Tom
Johan
Mathieu
Théo
Line

CPA1
05/11/25

Projet 1 :

Etape 2 : *Full StrongBox 3000*

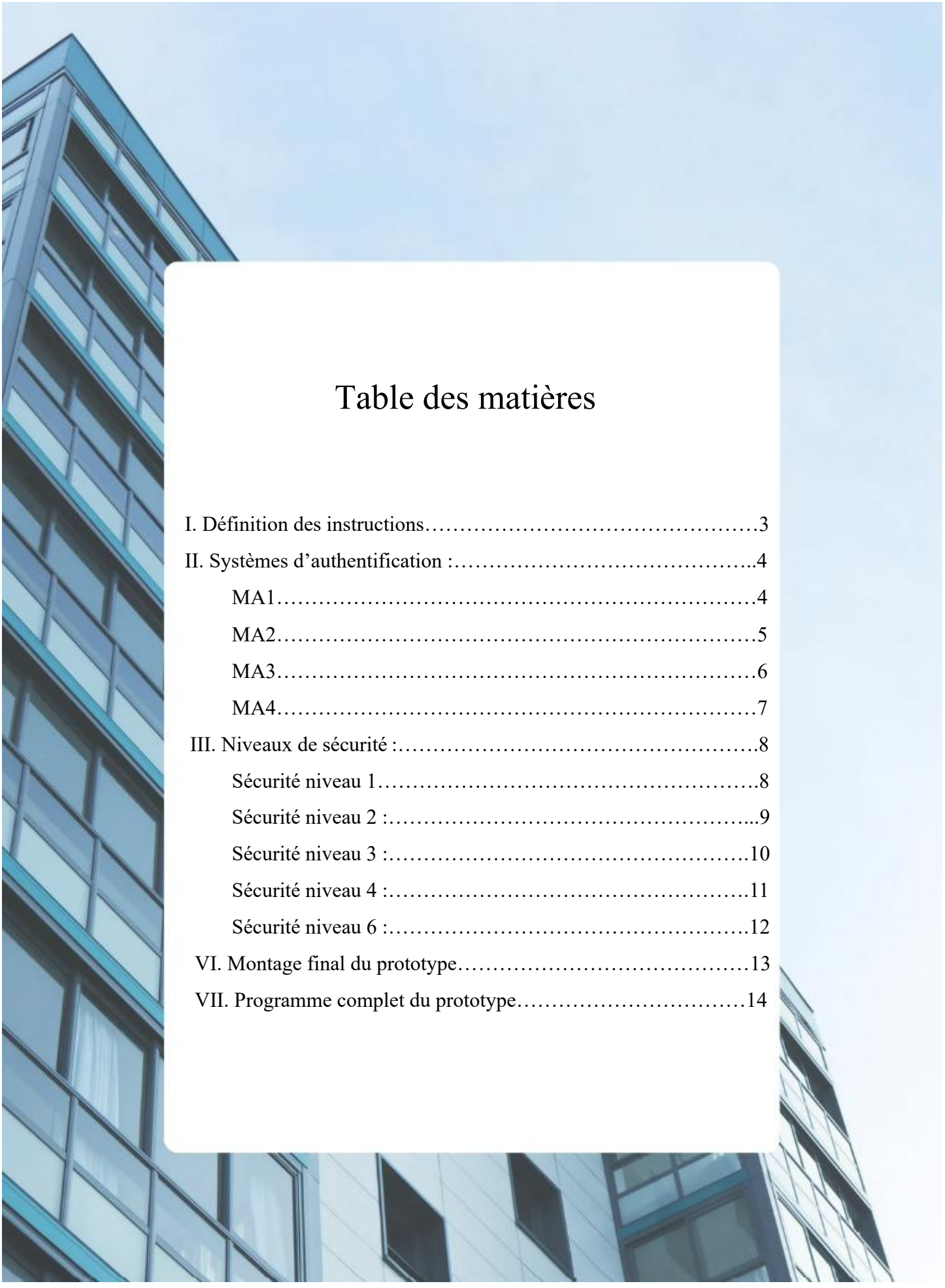


Table des matières

I. Définition des instructions……3

II. Systèmes d'authentification :……..4

MA1……4

MA2……5

MA3……6

MA4……7

III. Niveaux de sécurité :……8

Sécurité niveau 1……8

Sécurité niveau 2 :……...9

Sécurité niveau 3 :……10

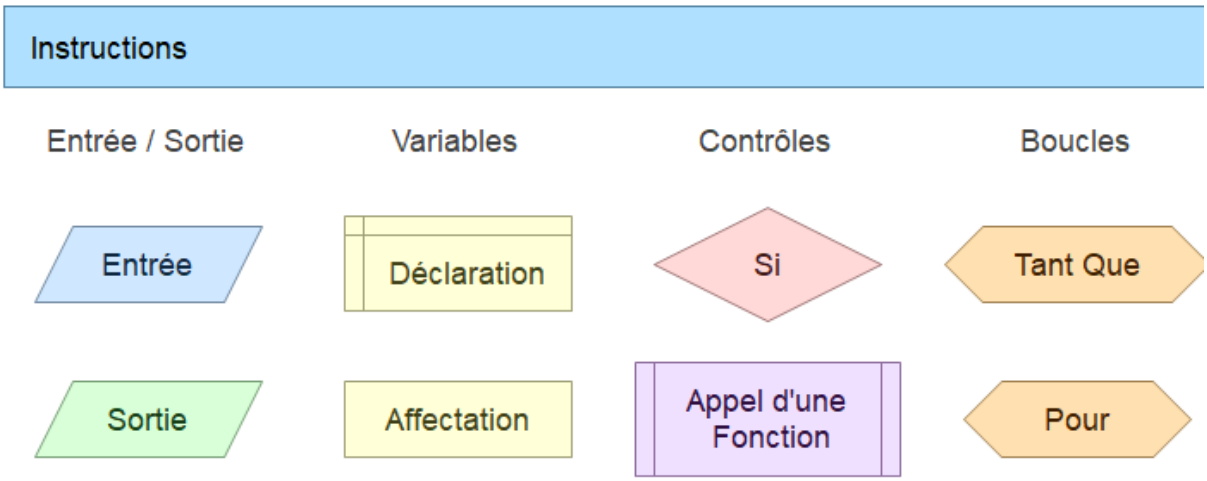
Sécurité niveau 4 :……11

Sécurité niveau 6 :……12

VI. Montage final du prototype……13

VII. Programme complet du prototype……14

I. Définition des instructions



Entrée / Sortie :

- **Entrée** : permet de recevoir une donnée de l'utilisateur, comme un nombre ou un texte (ex. : demander un âge).
- **Sortie** : affiche une information à l'écran, comme un résultat ou un message (ex. : afficher "Bonjour").

Variables :

- **Déclaration** : crée une variable, c'est-à-dire un espace mémoire pour stocker une donnée (ex. : int x).
- **Affectation** : donne une valeur à une variable (ex. : x = 5).

Contrôles :

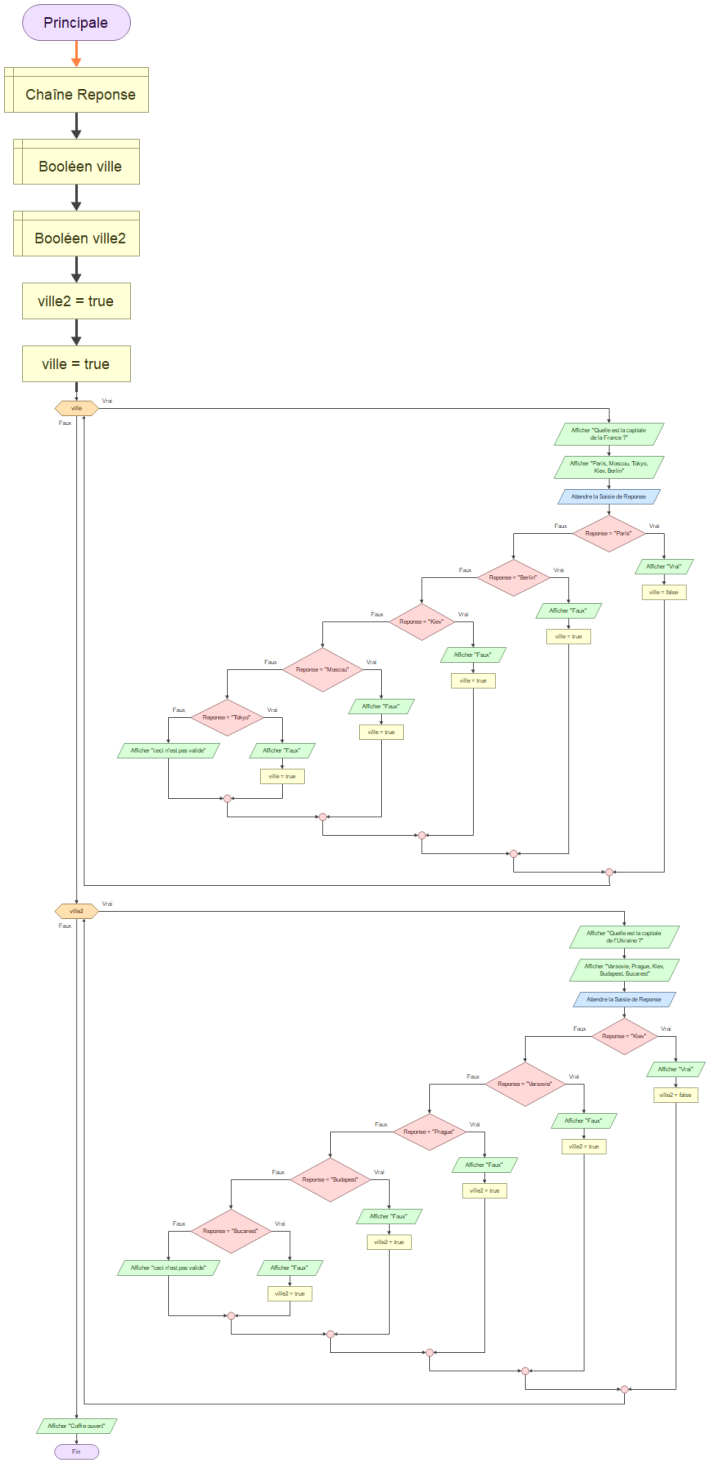
- **Si** : exécute une action seulement si une condition est vraie (ex. : si x > 10 alors...).

Boucles :

- **Tant que (orange, hexagone)** : répète une série d'instructions tant qu'une condition est vraie (ex. : tant que x < 10).

II. Systèmes d’authentification :

MA1



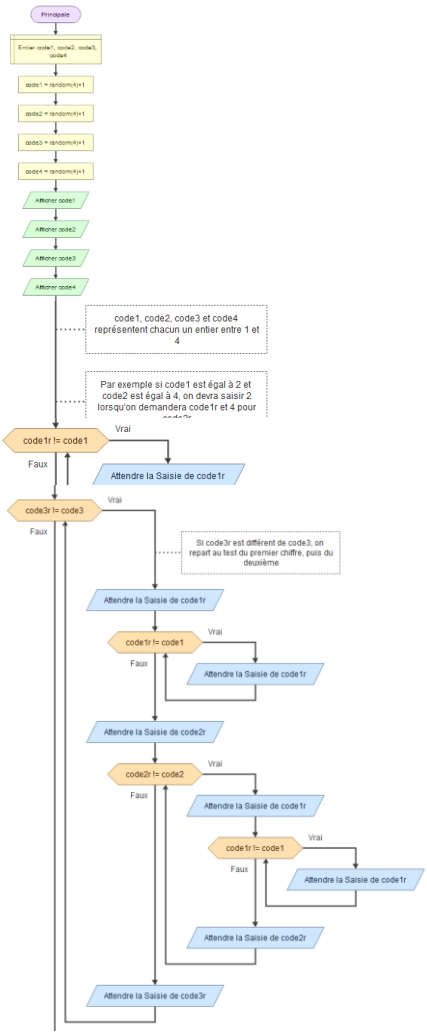
Ce schéma représente le fonctionnement d'un logigramme. Le programme commence par définir plusieurs variables : une variable pour la réponse de l'utilisateur et cinq chaînes de caractères contenant les mots « Paris », « Moscou », « Kiev », « Berlin » et « Tokyo ». Il initialise aussi deux variables booléennes appelées « ville » et « ville2 » à la valeur vraie.

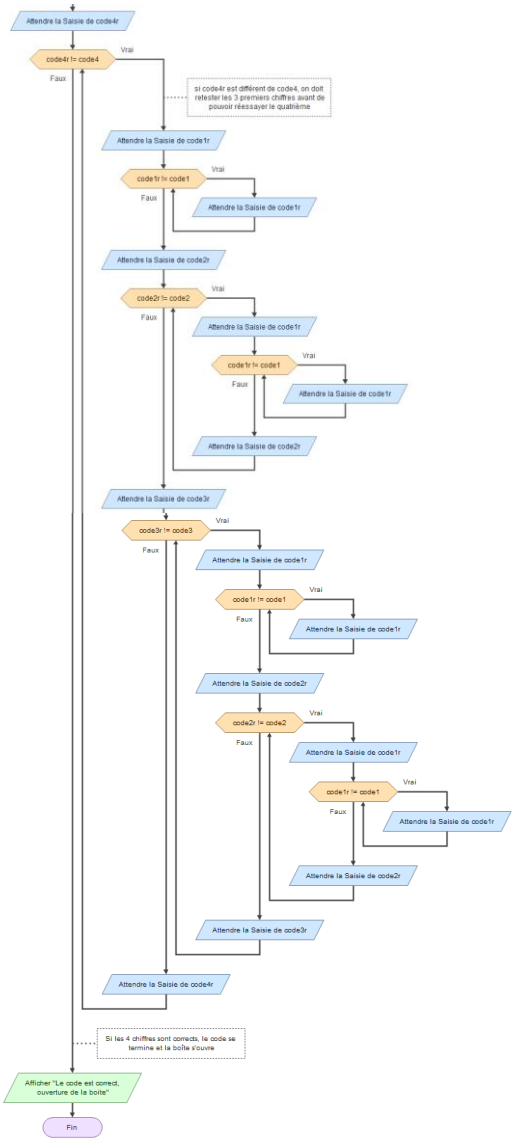
Tant que « ville » est vrai, le programme affiche la question « Quelle est la capitale de la France ? » ainsi que les cinq propositions : Paris, Moscou, Kiev, Berlin et Tokyo. Ensuite, il attend que l'utilisateur saisisse une réponse.

Quand la bonne réponse à « ville » est trouvée, on passe à une autre boucle pareille à la première : tant que « ville2 » est vrai, le programme affiche la question « Quelle est la capitale de l'Ukraine ? » ainsi que les cinq propositions : Varsovie, Prague, Kiev, Budapest, Bucarest. Lorsque la bonne réponse est trouvée, la console affiche alors « coffre ouvert ».

Quand une réponse est bonne, la console affiche « Vrai », et quand elle est fausse elle affiche « faux ». Lorsqu'une réponse est différente de celles attendues ou mal rédigée (ex : pas de majuscule…), la console affiche « Ceci n'est pas valide ».

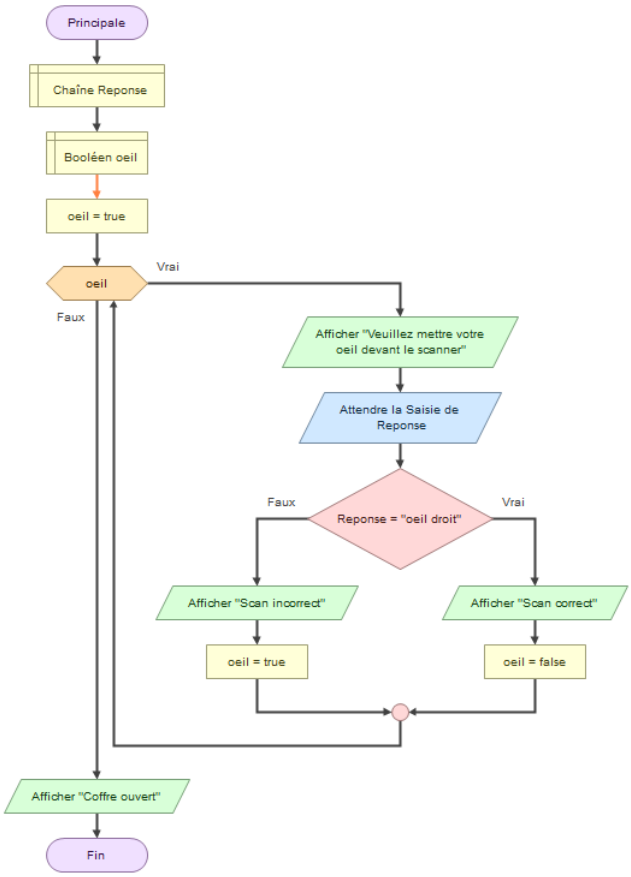
MA2





On a tout d'abord 4 variables : code1, code2, code3 et code4, représentant chacun un entier entre 1 et 4 (défini par « random(4) + 1 »). Ensuite, il y a 4 autres nouvelles variables : code1r, code2r, code3r, code4r. Ces 4 variables seront demandées à l'utilisateur l'une après l'autre. On commence donc par demander code1r. Si l'utilisateur donne le bon chiffre, on passe à code2r, et ainsi de suite. Cependant, si l'un des chiffres est incorrect, on recommencera à partir de code1r. Lorsque les 4 chiffres sont corrects la boite s'ouvre et affiche « Le code est correct, ouverture de la boite ».

MA3



Ce schéma représente un logigramme simple de vérification par scanner d'œil.

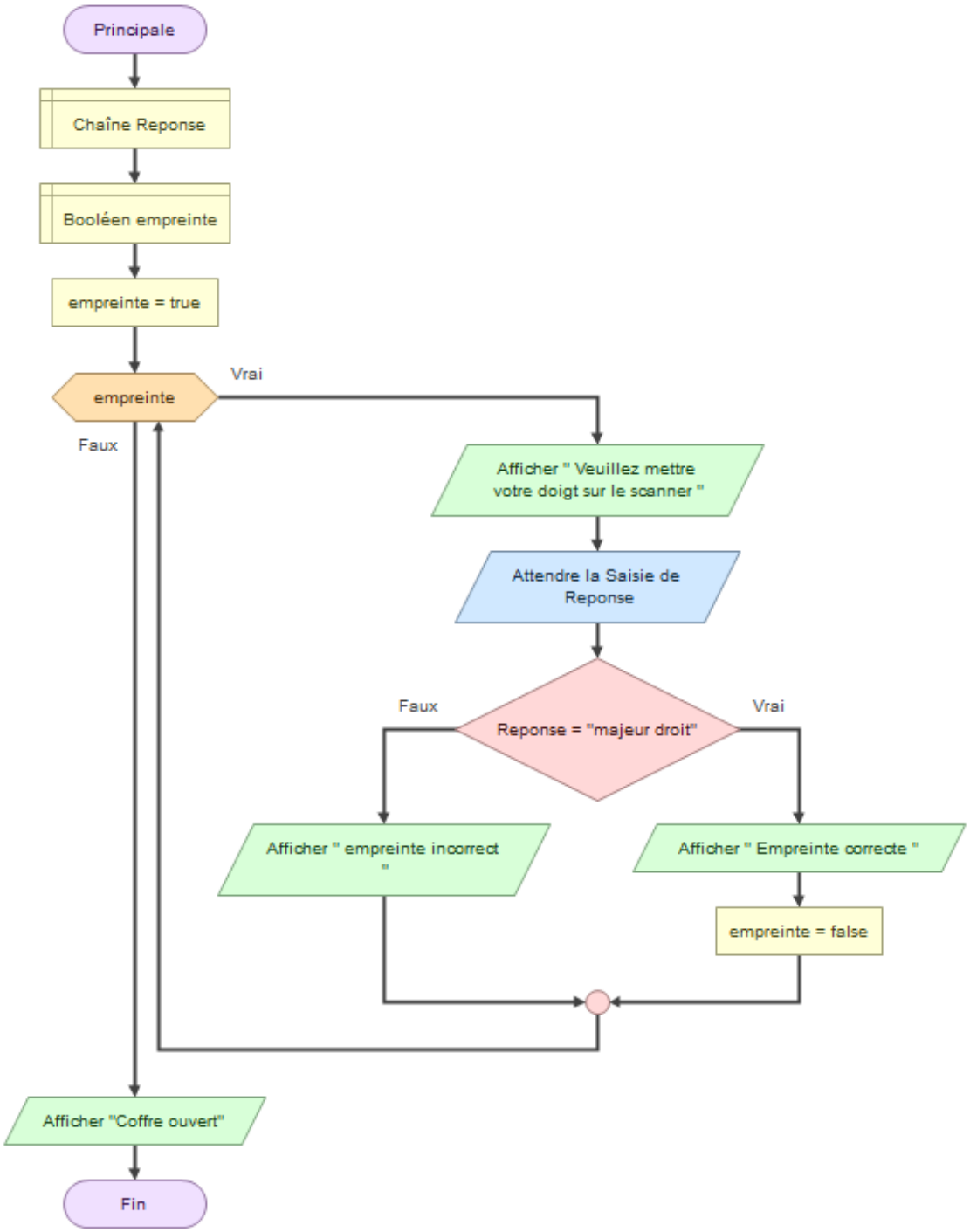
Le programme commence par le bloc principal et déclare une variable appelée « scan » de type chaîne de caractères. Ensuite, il affiche le message : « Veuillez mettre votre œil devant le scanner ».

Le programme attend ensuite la saisie de la variable « scan ». Une fois la saisie effectuée, il vérifie si la valeur de « scan » est égale à « œil droit ».

- Si la valeur saisie est « œil droit », le programme affiche « Scan correct ».
- Si la valeur est différente, il affiche « Scan incorrect ».

Après avoir affiché le message correspondant, le programme se termine.

MA4



Ce schéma représente un logigramme de vérification d'empreinte digitale.

Le programme commence par le bloc principal et affiche le message : « **Veuillez mettre votre doigt sur le scanner** ».
Ensuite, il déclare une variable appelée « **empreinte** » de type chaîne de caractères.

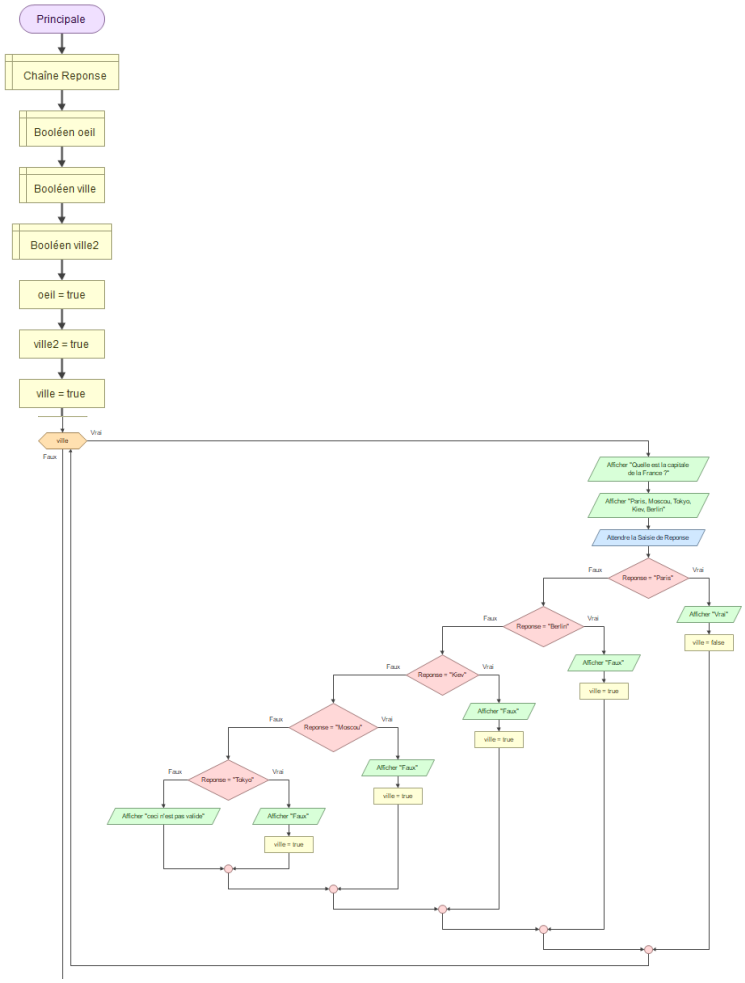
Le programme attend ensuite la **saisie de la variable empreinte**. Une fois la valeur saisie, il vérifie si **empreinte = "majeur droit"**.

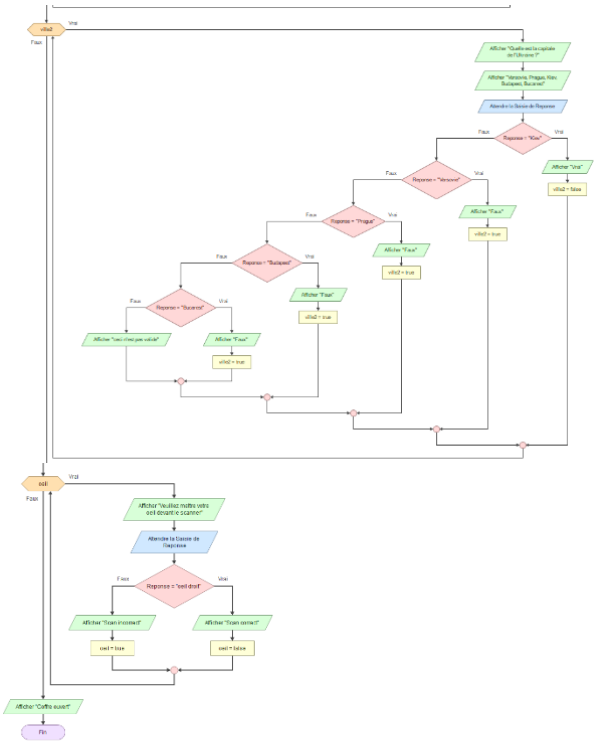
- Si la condition est vraie, le programme affiche « **Empreinte correcte** ».
- Si la condition est fausse, il affiche « **Empreinte incorrecte** ».

Enfin, après l'affichage du message correspondant, le programme se termine.

III. Niveaux de sécurité :

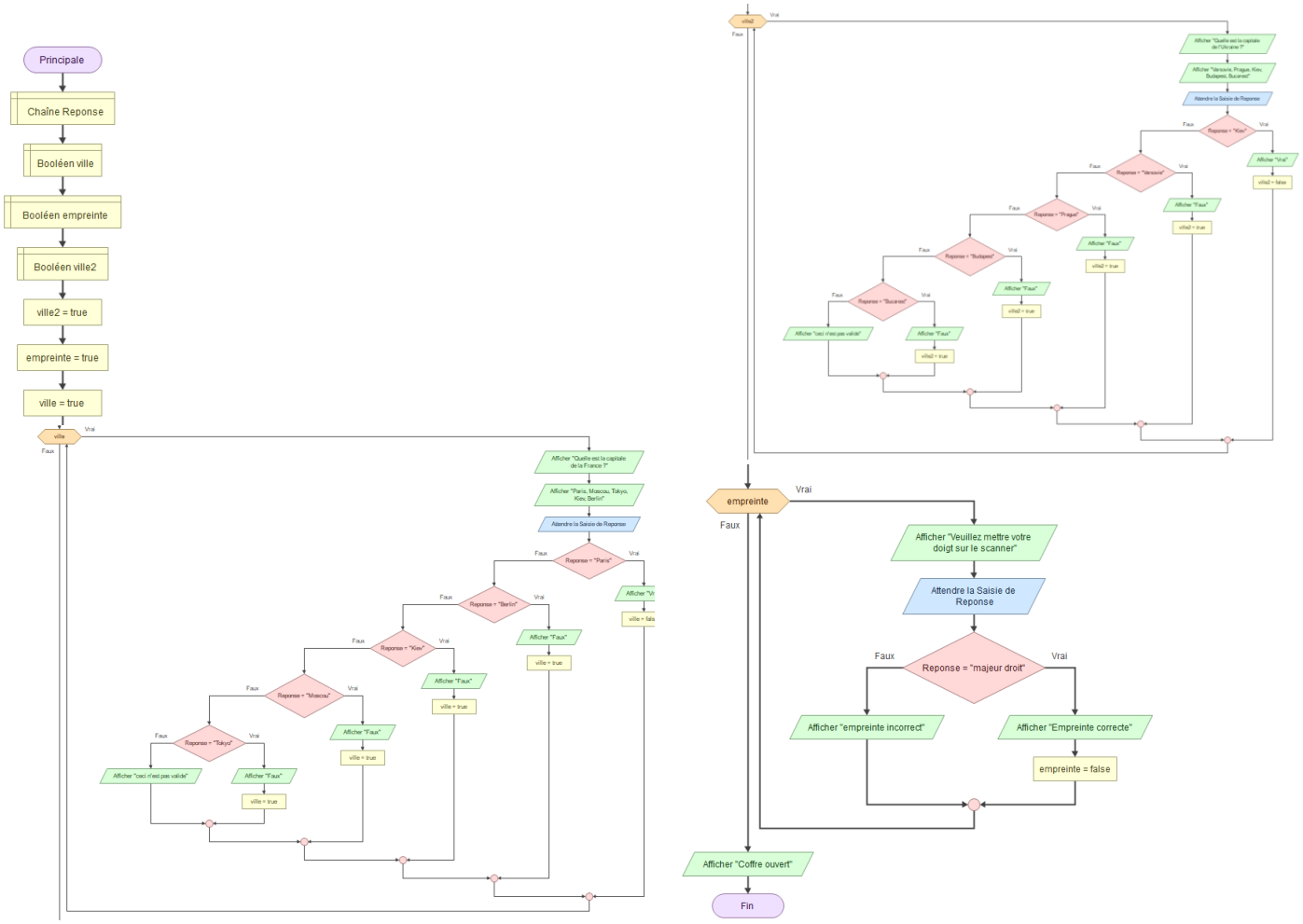
Sécurité niveau 1





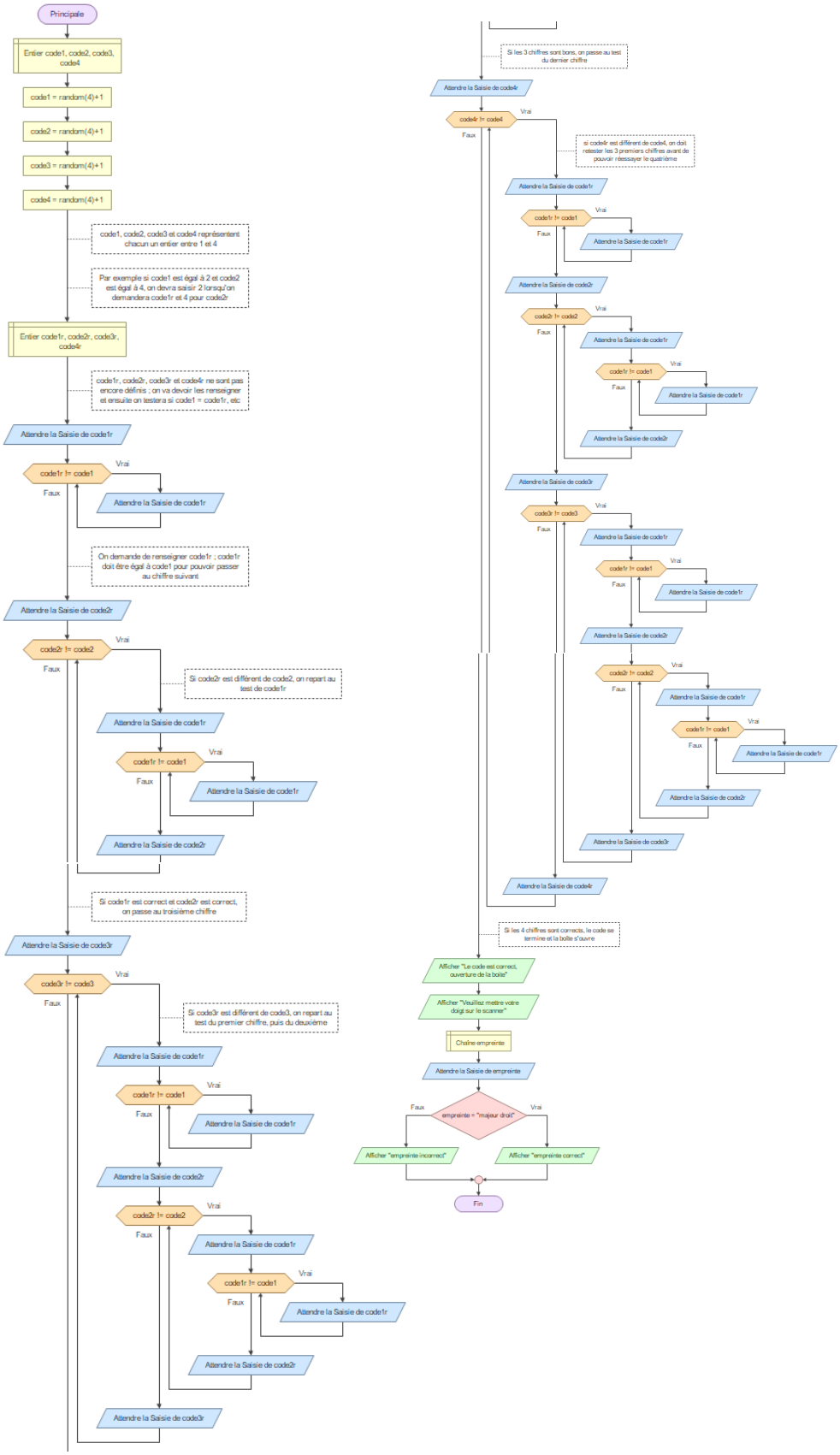
Ce logigramme est un assemblage de MA1 + MA3. Il représente une série de questions avec validation, si une réponse rendu est fausse, le programme nous renvoie à la première question. Si toutes les réponses saisis sont bonnes le programme nous demande le scan rétinien, s'il est correct le coffre est déverrouiller

Sécurité niveau 2 :



Ce logigramme est un assemblage de MA1 + MA4. Il représente une série de questions avec validation, si une réponse rendu est fausse, le programme nous renvoie à la première question. Si toutes les réponses saisis sont bonnes le programme nous demande le scan digital, s'il est correct le coffre est déverrouiller

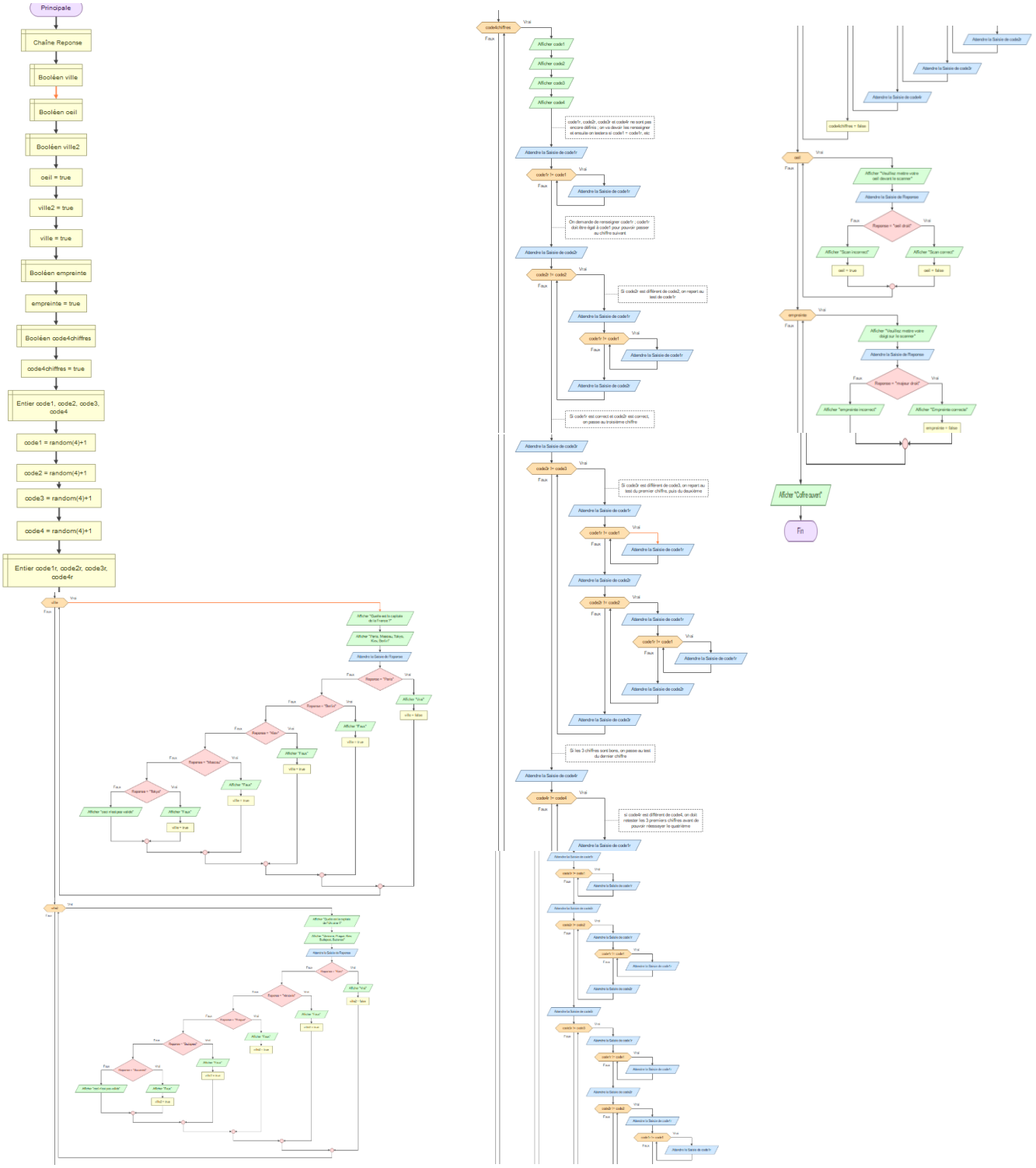
Sécurité niveau 3 :



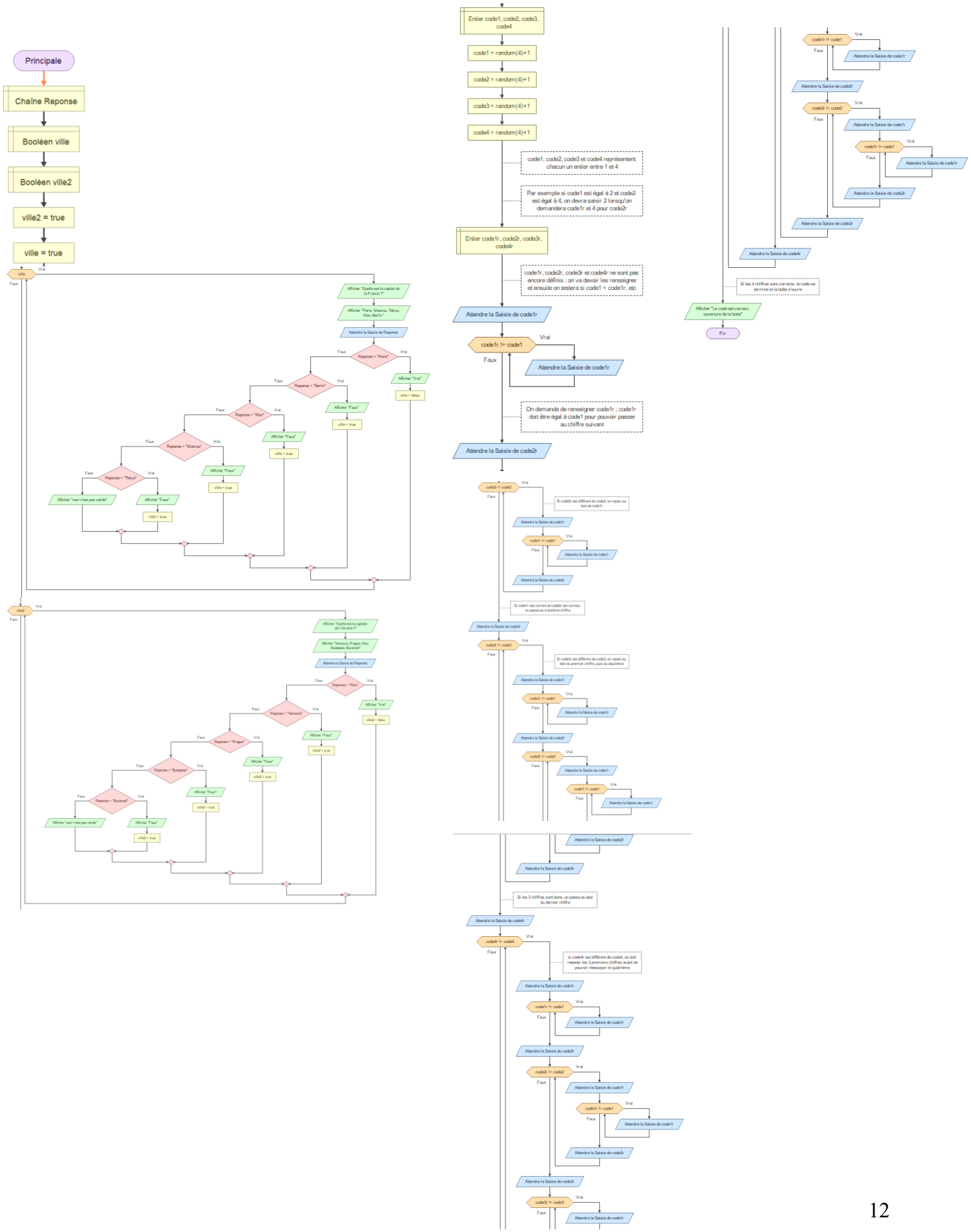
Ce logigramme est un assemblage de MA2 + MA4. Il représente génère un code random à 4 chiffres, avec chaque chiffre représentant un bouton poussoir. Si le code est rentré correctement, le programme nous demande le scan digital, s'il est correct le coffre est déverrouiller

Sécurité niveau 4 :

Ce logigramme est un assemblage de MA1 + MA2 + MA3 + MA4.

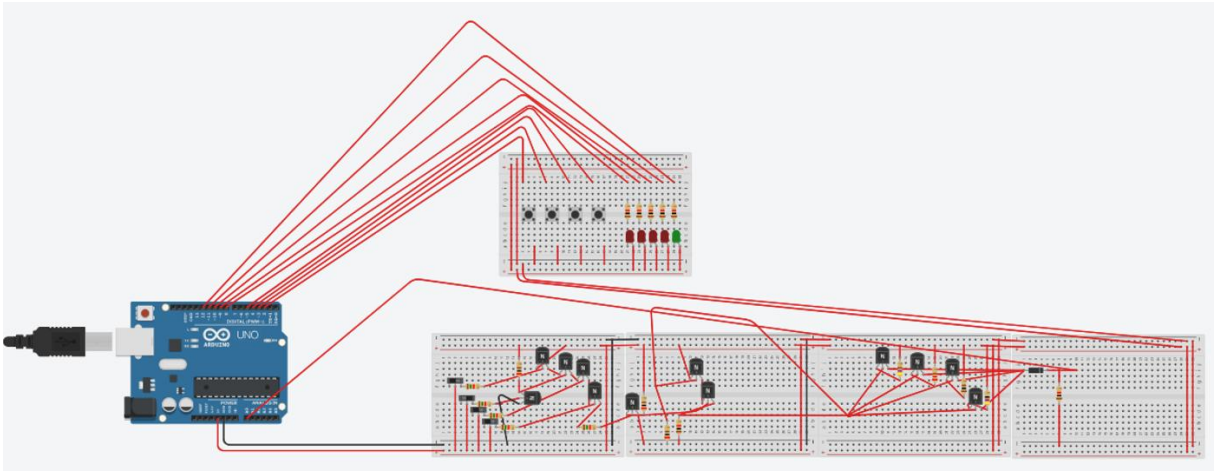


Sécurité niveau 6 :



Ce logigramme est un assemblage de MA1 + MA2 + MA5. Il représente représente une série de questions avec validation, si une réponse rendu est fausse, le programme nous renvoie à la première question. Si toutes les réponses saisis sont bonnes le programme nous demande un code random à 4 chiffres, avec chaque chiffre représentant un bouton poussoir. Si le code est rentré correctement, le coffre est déverrouillé.

VI. Montage final du prototype



VII. Programme complet du prototype

```
// ------------------------------
// SYSTEME DE SECURITE COMPLET
// ------------------------------
// Nom du programme.

// === PROTOTYPES ===
// Déclarations des fonctions utilisées dans le programme pour que le compilateur les connaisse.
bool questionSecurite();        // Etape 1 : Demande de réponses par le moniteur série.
bool sequenceBoutons();         // Etape 2 : Vérification du code secret entré par les boutons.
bool attendreScanRetinien();    // Etape 3 : Simulation de scan rétinien par le moniteur série.
bool verifierEmpreinteDigitale(); // Etape 4 : Simulation de vérification d'empreinte.
void eteindreToutesLesLeds();  // Éteint toutes les LEDs (Rouges et Verte).
void allumerLed(int pin, bool on); // Fonction générique pour allumer/éteindre une LED.
void stopSystem();              // Bloque le système en cas d'échec.
void clignoterLedVerte(int fois, int duree); // Fait clignoter la LED verte pour indiquer la réussite d'une étape.
int lireBouton();               // Lit l'état des boutons et gère l'anti-rebond/la détection de pression.
void genererCode();             // Génère le code aléatoire de l'étape 2.
bool verifierBoutonSimple(int bouton); // Vérifie si le bouton pressé correspond à la séquence attendue.

// === BROCHES ===
// Définition des broches numériques utilisées pour les LEDs et les Boutons.
const int ledRouge1 = 8;
const int ledRouge2 = 9;
const int ledRouge3 = 10;
const int ledRouge4 = 11;
const int ledVerte  = 12; // LED Verte, signal d'accès réussi ou d'étape validée.

const int bouton1 = 2;
const int bouton2 = 3;
const int bouton3 = 4;
const int bouton4 = 5;

// === VARIABLES GLOBALES ===
int codeSecret[4];          // Tableau pour stocker la séquence du code secret (4 chiffres).
int position = 0;           // Compteur de la position actuelle dans le code secret (0 à 3).
bool boutonPressse = false;  // Variable pour la gestion de l'anti-rebond des boutons (pour ne compter qu'une seule pression).

const bool LED_ACTIVE_HIGH = true;  // Détermine si HIGH allume la LED (True) ou LOW (False).

// Indicateurs de statut de chaque étape (initialement à false).
bool etape1_OK = false;
bool etape2_OK = false;
bool etape3_OK = false;
bool etape4_OK = false;

const int MAX_ATTEMPTS_CODE = 2; // Nombre maximum d'erreurs autorisées pour l'Étape 2 (Code Secret).
const int MAX_ATTEMPTS_AUTRES = 1; // Nombre maximum d'erreurs pour les autres étapes (ici, une seule tentative).

// ------------------------------
// SETUP
// ------------------------------
void setup() {
  Serial.begin(9600); // Initialisation de la communication série à 9600 bauds.
  Serial.println("--- SYSTEME DE SECURITE ACTIVE ---");

  // Configuration des broches des LEDs en mode SORTIE (OUTPUT).
  pinMode(ledRouge1, OUTPUT);
  pinMode(ledRouge2, OUTPUT);
  pinMode(ledRouge3, OUTPUT);
  pinMode(ledRouge4, OUTPUT);
  pinMode(ledVerte, OUTPUT);

  // Configuration des broches des boutons en mode ENTREE avec résistance PULLUP interne.
  // Un bouton pressé mettra la broche à LOW.
  pinMode(bouton1, INPUT_PULLUP);
  pinMode(bouton2, INPUT_PULLUP);
  pinMode(bouton3, INPUT_PULLUP);
  pinMode(bouton4, INPUT_PULLUP);

  eteindreToutesLesLeds(); // S'assure que toutes les LEDs sont éteintes au démarrage.
  randomSeed(analogRead(A0)); // Initialise le générateur de nombres aléatoires avec une source analogique (pour le code secret).

  // ---- Début des étapes de sécurité ----

  // === ETAPE 1 === (Questions de sécurité)
  if (!questionSecurite()) stopSystem(); // Si l'étape échoue, appel à stopSystem.
  Serial.println("ETAPE 1 REUSSIE");
  clignoterLedVerte(2, 200); // Signal de réussite.

  // === ETAPE 2 === (Code secret par boutons)
  if (!sequenceBoutons()) stopSystem();
  Serial.println("ETAPE 2 REUSSIE");
  clignoterLedVerte(2, 200);

  // === ETAPE 3 === (Scan rétinien)
  if (!attendreScanRetinien()) stopSystem();
  Serial.println("ETAPE 3 REUSSIE");
  clignoterLedVerte(2, 200);

  // === ETAPE 4 === (Empreinte digitale)
  if (!verifierEmpreinteDigitale()) stopSystem();
  Serial.println("ETAPE 4 REUSSIE");
  clignoterLedVerte(2, 200);

  // Séquence de réussite finale
  allumerLed(ledVerte, true);
  Serial.println("ACCES AUTORISE - SEQUENCE TERMINEE");
}

void loop() {} // La fonction loop est vide car toute la logique de sécurité est exécutée dans setup().

// ------------------------------
// FONCTIONS UTILES
// ------------------------------

// Fonction pour contrôler l'état d'une LED en tenant compte de la configuration ACTIVE_HIGH/LOW.
void allumerLed(int pin, bool on) {
  if (LED_ACTIVE_HIGH) digitalWrite(pin, on ? HIGH : LOW);
  else digitalWrite(pin, on ? LOW : HIGH);
}

// Fonction pour éteindre toutes les LEDs.
void eteindreToutesLesLeds() {
  allumerLed(ledRouge1, false);
  allumerLed(ledRouge2, false);
  allumerLed(ledRouge3, false);
  allumerLed(ledRouge4, false);
  allumerLed(ledVerte, false);
}

// Fonction pour bloquer le système en cas d'échec.
void stopSystem() {
  // Allume toutes les LEDs Rouges pour signaler le blocage.
  allumerLed(ledRouge1, true);
  allumerLed(ledRouge2, true);
  allumerLed(ledRouge3, true);
  allumerLed(ledRouge4, true);
  allumerLed(ledVerte, false);
  Serial.println("SYSTEME BLOQUE : une authentification a echoue. ARRET.");
  while(true){} // Boucle infinie, bloque le programme (l'Arduino ne fait plus rien).
}

// Fonction pour faire clignoter la LED verte.
void clignoterLedVerte(int fois, int duree) {
  for(int i=0; i<fois; i++){
    allumerLed(ledVerte, true);
    delay(duree);
    allumerLed(ledVerte, false);
    delay(duree);
  }
}

// ------------------------------
// ETAPE 1 : QUESTIONS DE SECURITE
// ------------------------------
bool questionSecurite() {
  Serial.println("\n[ETAPE 1] QUESTIONS DE SECURITE :");

  //    Question 1 : France
  Serial.println("Question 1 : Quelle est la capitale de la France ?");
  Serial.println("1) Moscou\n2) Paris\n3) Berlin\n4) Tokyo\n5) Kiev");
  Serial.print("Tapez votre réponse : ");

  while (Serial.available() == 0) {} // Attend que l'utilisateur tape quelque chose dans le moniteur série.
  String reponse1 = Serial.readStringUntil('\n'); // Lit l'entrée jusqu'au retour à la ligne.
  reponse1.trim(); // Supprime les espaces inutiles.

  // Vérifie si la réponse est "Paris" ou "2" (insensible à la casse pour "Paris").
  if (!(reponse1.equalsIgnoreCase("Paris") || reponse1.equals("2"))) {
    Serial.println("Mauvaise réponse à la question 1. Tentative unique dépassée.");
    return false;
  }

  Serial.println("Bonne réponse à la question 1 !");

  // --- Question 2 : Ukraine ---
  Serial.println("\nQuestion 2 : Quelle est la capitale de l'Ukraine ?");
  Serial.println("1) Varsovie\n2) Prague\n3) Kiev\n4) Budapest\n5) Bucarest");
  Serial.print("Tapez votre réponse : ");

  while (Serial.available() == 0) {} // Attend la deuxième réponse.
  String reponse2 = Serial.readStringUntil('\n');
  reponse2.trim();

  // Vérifie la deuxième réponse.
  if (reponse2.equalsIgnoreCase("Kiev") || reponse2.equals("3")) {
    Serial.println("Bonne réponse à la question 2 !");
    etape1_OK = true;
    return true; // Étape réussie.
  } else {
    Serial.println("Mauvaise réponse à la question 2. Tentative unique dépassée.");
    return false; // Étape échouée.
  }
}

// ------------------------------
// ETAPE 2 : CODE SECRET
// ------------------------------
bool sequenceBoutons() {
  Serial.println("\n[ETAPE 2] CODE SECRET :");
  genererCode(); // Génère le nouveau code secret.
  position = 0;
  eteindreToutesLesLeds();

  int wrongCount = 0; // Compteur d'erreurs pour cette étape.

  // Boucle tant que le code n'est pas entièrement entré (position < 4) et que le max d'erreurs n'est pas atteint.
  while (position < 4 && wrongCount < MAX_ATTEMPTS_CODE) {
    int bouton = lireBouton(); // Lit si un bouton a été pressé (retourne 1, 2, 3, 4 ou 0).

    if (bouton != 0) { // Si un bouton a été pressé

      if (verifierBoutonSimple(bouton)) { // Vérifie si le bouton correspond à la position actuelle.

        // Si la vérification est réussie et que la position atteint 4 (code complet).
        if (position == 4) {
          etape2_OK = true;
          allumerLed(ledVerte, true);
          return true; // Étape réussie.
        }
      } else {
        // Erreur de séquence
        wrongCount++;
        Serial.println("Erreur sur ETAPE 2 !");
        position = 0; // Réinitialise la séquence
        eteindreToutesLesLeds();

        // Vérifie si le max d'erreurs est atteint.
        if (wrongCount >= MAX_ATTEMPTS_CODE) {
          Serial.println("Nombre max d'erreurs dépassé pour ETAPE 2.");
          return false; // Étape échouée.
        }
      }
    }
  }
  // Si on sort de la boucle sans succès (p.ex. par manque de saisie), on échoue.
  return false;
}

// Fonction pour lire l'état des boutons.
int lireBouton() {
  // Lit les boutons. S'il est à LOW (pressé) ET qu'aucun bouton n'était déjà en cours de pression (!boutonPresse),
  // on marque le bouton comme pressé, et on retourne son identifiant.
  if (digitalRead(bouton1) == LOW && !boutonPresse){ boutonPresse=true; return 1; }
  if (digitalRead(bouton2) == LOW && !boutonPresse){ boutonPresse=true; return 2; }
  if (digitalRead(bouton3) == LOW && !boutonPresse){ boutonPresse=true; return 3; }
  if (digitalRead(bouton4) == LOW && !boutonPresse){ boutonPresse=true; return 4; }

  // Si TOUS les boutons sont relâchés (HIGH), on réinitialise le drapeau boutonPresse.
  // C'est la logique d'anti-rebond basique qui permet de détecter la prochaine pression.
  if (digitalRead(bouton1)==HIGH && digitalRead(bouton2)==HIGH && digitalRead(bouton3)==HIGH && digitalRead(bouton4)==HIGH){
    boutonPresse=false;
  }
  return 0; // Rien de nouveau n'est pressé.
}

// Génère un nouveau code secret aléatoire.
void genererCode() {
  position = 0;
  Serial.print("Code secret (affiché pour l'utilisateur) : ");
  for (int i=0;i<4;i++){
    codeSecret[i]=random(1,5); // Génère un nombre aléatoire entre 1 et 4 (inclus).
    Serial.print(codeSecret[i]);
    if(i<3) Serial.print("-");
  }
  Serial.println("\nAppuie sur les boutons dans cet ordre !");
}

// Vérifie si le bouton pressé correspond au chiffre attendu dans la séquence.
bool verifierBoutonSimple(int bouton){
  if (bouton==codeSecret[position]) { // Si le bouton est correct
    // Allume la LED rouge correspondante à la position actuelle.
    if(position==0) allumerLed(ledRouge1,true);
    if(position==1) allumerLed(ledRouge2,true);
    if(position==2) allumerLed(ledRouge3,true);
    if(position==3) allumerLed(ledRouge4,true);
    position++; // Passe à la position suivante du code.
    return true;
  } else {
    // Si le bouton est incorrect
    position=0; // Réinitialise la séquence complète.
    eteindreToutesLesLeds(); // Éteint les LEDs rouges déjà allumées.
    return false;
  }
}

// ------------------------------
// ETAPE 3 : SCAN RETINIEN (Simulé)
// ------------------------------
bool attendreScanRetinien() {
  Serial.println("\n[ETAPE 3] SCAN RETINIEN :");
  // Boucle de tentative (ici limitée à une seule dans le setup)
  while(!etape3_OK){
    Serial.print("Placez votre oeil et tapez la reponse correcte : ");
    while (Serial.available() == 0) {}
    String entree = Serial.readStringUntil('\n'); entree.trim();

    if (entree.equalsIgnoreCase("oeil droit")){ // Condition de succès (simulation).
      etape3_OK=true;
      return true;
    } else {
      Serial.println("Erreur sur ETAPE 3. Tentative unique depassee.");
      return false; // Échoue après une seule tentative.
    }
  }
  return false; // Ne devrait pas être atteint dans ce setup.
}

// ------------------------------
// ETAPE 4 : EMPREINTE DIGITALE (Simulée)
// ------------------------------
bool verifierEmpreinteDigitale(){
  Serial.println("\n[ETAPE 4] EMPREINTE DIGITALE :");
  // Boucle de tentative (ici limitée à une seule dans le setup)
  while(!etape4_OK){
    Serial.print("Placez votre doigt et tapez la reponse correcte : ");
    while (Serial.available() == 0) {}
    String entree = Serial.readStringUntil('\n');
    entree.trim();

    if(entree.equalsIgnoreCase("majeur droit")){ // Condition de succès (simulation).
      etape4_OK = true;
      return true;
    } else {
      Serial.println("Erreur sur ETAPE 4. Tentative unique depassee.");
      return false; // Échoue après une seule tentative.
    }
  }
  return false; // Ne devrait pas être atteint dans ce setup.
}
```

VIII. Conclusion :

En conclusion nous pouvons dire que nous avons bien respecté le cahier des charges. Nous avons répondu aux attentes et contraintes en temps et en heure et nous avons pu rendre un dossier complet comprenant notre travail finalisé.